



جامعة دمشق
كلية الهندسة المعلوماتية
السنة الرابعة - شبكات

برمجة تفرعية

إعداد الطلاب :

مصطفى جلال غانم

عبدالله محمد أنس شيخ الأرض

أحمد محمد بشار زيتون

أمجد أسامة أبو بكر

إشراف المهندس : حذيفة عقيل

الطلب الاول : حماية البيانات المشتركة من التضارب.

```
Body Cookies (1) Headers (11) Test Results
{ } JSON Preview Visualize
1 {
2   "message": "Product added",
3   "data": {
4     "brand": "brand1",
5     "category": {
6       "id": 1,
7       "name": "string"
8     },
9     "description": "any description",
10    "id": 4,
11    "images": [],
12    "inventory": 1,
13    "name": "product1",
14    "price": 3380.7971958595617
15  }
16 }
```

اضافة منتج و وضع كميته في المستودع تساوي 1

اضافة المنتج الى سلة اثنين من المستخدمين

POST `{{baseUrl}} /api/v1/cartItems/item/add?productId=4&quantity=1`

Docs Params Authorization Headers (11) Body Scripts Tests Settings

Query Params

☒	Key	Value
☒	productId *	integer 4
☒	quantity *	integer 1

Result Grid Filter Rows: Edit: Export/Import:

	id	brand	description	inventory	name	price	category_id
▶	4	brand1	any description	1	product1	3380.80	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

التأكد من الكمية في قاعدة البيانات

تم اتمام الطلب عند المستخدمين

race condition - Run results

Ran today at 09:00:26 AM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	1	715ms	0	0	71 ms

All Tests Passed (0) Failed (0) Skipped (0) Errors (0) Console Log [View Summary](#)

Iteration 1

POST create Order user1
http://localhost:8080/api/v1/orders/order?userId=1 200 • No tests found

POST create Order user2
http://localhost:8080/api/v1/orders/order?userId=2 200 • No tests found

1 POST race condition / create Order user1

Response Headers Request 200 • 74 ms • 564 B

Pretty

```
1 {
2   "message": "Order created successfully",
3   "data": {
4     "id": 3,
5     "items": [
6       {
7         "price": 3380.80,
8         "productId": 4,
9         "productName": "product1",
10        "quantity": 1
11      }
12    ],
13    "orderDate": "2026-05-06T09:00:27.3763434",
14    "orderId": 3,
15    "status": "PENDING",
16    "totalPrice": 3380.80
17  }
18 }
```

race condition - Run results

Ran today at 09:00:26 AM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	1	715ms	0	0	71 ms

All Tests Passed (0) Failed (0) Skipped (0) Errors (0) Console Log [View Summary](#)

Iteration 1

POST create Order user1
http://localhost:8080/api/v1/orders/order?userId=1 200 • No tests found

POST create Order user2
http://localhost:8080/api/v1/orders/order?userId=2 200 • No tests found

1 POST race condition / create Order user2

Response Headers Request 200 • 68 ms • 564 B

Pretty

```
1 {
2   "message": "Order created successfully",
3   "data": {
4     "id": 4,
5     "items": [
6       {
7         "price": 3380.80,
8         "productId": 4,
9         "productName": "product1",
10        "quantity": 1
11      }
12    ],
13    "orderDate": "2026-05-06T09:00:27.5233443",
14    "orderId": 4,
15    "status": "PENDING",
16    "totalPrice": 3380.80
17  }
18 }
```

Result Grid Filter Rows: Edit Export/Import

	id	brand	description	inventory	name	price	category_id
▶	4	brand1	any description	-1	product1	3380.80	1
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL

التأكد من المشكلة في قاعدة البيانات

الحل :

```
@Transactional // entire order is one atomic transaction
public Order placeOrder(Long userId) {

    // Locks the product row until transaction completes
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Product p WHERE p.id = :id")
    Optional<Product> findByIdWithLock(@Param("id") Long id);

    // Atomic decrement - only succeeds if enough stock exists
    @Modifying
    @Query("UPDATE Product p SET p.inventory = p.inventory -
:quantity " +
        "WHERE p.id = :id AND p.inventory >= :quantity")
    int decreaseInventory(@Param("id") Long id,
        @Param("quantity") int quantity);
```

نتيجة الحل :

race condition - Run results

Run today at 11:15:57 AM • View all runs

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	New Environment	1	798ms	0	0	91 ms

All Tests Passed (0) Failed (0) Skipped (0) Errors (0) Console Log View Summary

Iteration 1

POST create Order user1
http://localhost:8080/api/v1/orders/order?userId=1 200 •
No tests found

POST create Order user2
http://localhost:8080/api/v1/orders/order?userId=2 500 •
No tests found

1 POST race condition / create Order user1

Response Headers Request

Pretty

```
1 {
2   "message": "Order created successfully",
3   "data": {
4     "id": 5,
5     "items": [
6       {
7         "price": 3388.88,
8         "productId": 4,
9         "productName": "product1",
10        "quantity": 1
11      }
12    ],
13    "orderDate": "2026-05-06T11:15:58.1562313",
14    "orderId": 5,
15    "status": "PENDING",
16    "totalPrice": 3388.88
17  }
18 }
```

فقط مستخدم واحد تم قبول طلبه اما الاخر تم ارجاع status code 500 مع رسالة خطأ واضحة

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
HTTP Reque...	20	109	18	408	131.23	95.00%	20.6/sec	112.21	8.55	5567.1
TOTAL	20	109	18	408	131.23	95.00%	20.6/sec	112.21	8.55	5567.1

☐ Include group name in label? ☒ Save Table Header

تم إعادة اختبار النظام باستخدام:

Apache JMeter

وإرسال:

20 Concurrent Requests

النتائج النهائية

• طلب واحد فقط نجح :

200 OK

• باقي الطلبات فشلت برسالة :

Out of stock

الاستنتاج

تم حل مشكلة:

Race Condition

بنجاح باستخدام:

• Pessimistic Locking

• Transaction Management

• Inventory Validation

• Cart Locking

الطلب الثاني + الثالث : إدارة الموارد والمعالجة غير المتزامنة (Resource Management & Async Processing)

1. الهدف :

تحسين أداء واستجابة النظام عبر نقل المهام الثقيلة (مثل إرسال الإشعارات) خارج المسار الرئيسي للطلب، إدارتها باستخدام مجمع خيوط (Thread Pool) محدد لضمان استقرار السيرفر تحت الضغط العالي.

2. الاستراتيجية المستخدمة Thread Pool Executor :

- وصف الاستراتيجية: تم إعداد مجمع خيوط مخصص (Custom Thread Pool) يحتوي على عدد محدد من العمال (Core Pool Size: 5) وطابور انتظار (Queue).
- المعالجة غير المتزامنة (@Async): تم استخدام الوسم @Async لفصل المهام الجانبية عن عملية "إنشاء الطلب" الأساسية، مما يسمح برد سريع للمستخدم بينما تستمر المعالجة في الخلفية.

```
package com.shoppingCartBackend.shoppingCartBackend.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;
import java.util.concurrent.Executor;

@Configuration
public class AsyncConfig {

    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();

        executor.setCorePoolSize(5); // الأساسي Threads عدد
        executor.setMaxPoolSize(10); // الحد الأقصى
        executor.setQueueCapacity(50); // عدد الطلبات المنتظرة

        executor.setThreadNamePrefix("Ecommerce-Thread-");
        executor.initialize();

        return executor;
    }
}
```

"شكل (1): إعدادات مجمع الخيوط المخصص وتحديد سعة طابور الانتظار لضمان إدارة الموارد".

3. تبرير الاختيار :

- سرعة الاستجابة (Latency): تقليل وقت انتظار المستخدم من عدة ثوانٍ إلى أجزاء من الثانية.
- حماية الذاكرة: منع النظام من فتح خيوط لا نهائية (Threads) قد تؤدي لانهايار السيرفر، والالتزام بحدود الـ Core Pool المحددة بـ 5 فقط.
- استمرارية العمل: في حال فشل المهمة الخلفية (الإشعار)، لا يتأثر نجاح عملية الشراء الأساسية.

4. التصميم المعماري (الرسم التخطيطي) :

[Clients] -> [OrderController : 8080]

(Primary Path - Sync) -> [Database: Save Order]

(Side Path - Async) -> [TaskExecutor (Thread Pool)]

[Ecommerce-Thread 1-5] -> [Notification Service]

5. التجربة والنتائج :

a. إعدادات التجربة:

• عدد المهام الخلفية: إرسال إشعار لكل طلب (محاكاة تأخير 5 ثوانٍ).

• عدد الطلبات المرسل: 10 طلبات متزامنة عبر Postman Runner.

b. النتائج:

• متوسط وقت الاستجابة للمستخدم (20ms - 60ms): بدلاً من 50 ثانية في الحالة العادية .

• نجاح توزيع المهام بنسبة 100% دون أخطاء.

The screenshot displays the 'Run results' for a collection named 'My Collection'. It shows a single run at 02:13:47 PM. The summary table indicates 10 iterations, a total duration of 1s 518ms, 0 failed tests, and 0 errors. The average response time is 45 ms. Below the summary, the details of the first iteration are shown: a POST request to 'http://localhost:8080/api/v1/orders/order' with a 200 status code, 61 ms response time, and 563 B of data. The second and third iterations are also shown, with response times of 53 ms and 47 ms respectively, all with 200 status codes.

Source	Environment	Iterations	Duration	All tests	Errors
Runner	none	10	1s 518ms	0	0

Avg. Resp. Time: 45 ms

All Tests: Passed (0) Failed (0) Skipped (0) Errors (0) Console Log View Summary

Iteration 1: POST http://localhost:8080/api/v1/orders/order. Status: 200, 61 ms, 563 B.

Iteration 2: POST http://localhost:8080/api/v1/orders/order. Status: 200, 53 ms, 563 B.

Iteration 3: POST http://localhost:8080/api/v1/orders/order. Status: 200, 47 ms, 563 B.

"شكل (3): نتائج اختبار الأداء عبر Postman تظهر سرعة استجابة النظام (متوسط 60ms) للطلبات المتزامنة".

6. تحليل السجلات (Logs Analysis) :

تظهر السجلات أن الخيوط من Ecommerce-Thread-1 إلى Ecommerce-Thread-5 استلمت المهام بالتبادل وبمجرد انتهاء خيط من مهمة، يقوم بسحب المهمة التالية من الطابور (Thread Reuse) .

```
Finished Notification for Order: 155 | Thread: Ecommerce-Thread-2
Started Notification for Order: 160 | Thread: Ecommerce-Thread-2
Finished Notification for Order: 156 | Thread: Ecommerce-Thread-3
Started Notification for Order: 161 | Thread: Ecommerce-Thread-3
Finished Notification for Order: 157 | Thread: Ecommerce-Thread-4
Started Notification for Order: 162 | Thread: Ecommerce-Thread-4
Finished Notification for Order: 158 | Thread: Ecommerce-Thread-5
Started Notification for Order: 163 | Thread: Ecommerce-Thread-5
Finished Notification for Order: 159 | Thread: Ecommerce-Thread-1
Started Notification for Order: 164 | Thread: Ecommerce-Thread-1
Finished Notification for Order: 160 | Thread: Ecommerce-Thread-2
Finished Notification for Order: 161 | Thread: Ecommerce-Thread-3
Finished Notification for Order: 162 | Thread: Ecommerce-Thread-4
Finished Notification for Order: 163 | Thread: Ecommerce-Thread-5
Finished Notification for Order: 164 | Thread: Ecommerce-Thread-1
```

"شكل (4): سجلات النظام تثبت توزيع المهام على خيوط المعالجة الخلفية وإعادة استخدامها بكفاءة".

الطلب الرابع : معالجة البيانات الضخمة على دفعات (Batch Processing)

المطلوب تنفيذ جرد للمبيعات اليومية .

مقارنة الأداء قبل وبعد استخدام :

- Batch Processing
- Chunk Processing
- Background Processing
- Async Processing

المرحلة الأولى: التنفيذ بدون Chunking

الفكرة

في البداية تم تنفيذ جرد المبيعات بشكل متسلسل Sequential Processing. أي أن النظام:

- يقرأ الطلبات واحدًا تلو الآخر .
- ينفذ المعالجة الثقيلة لكل Order.
- ينتظر انتهاء الطلب الحالي قبل الانتقال للذي بعده .

```
// BEFORE
// =====|
public BatchResultResponse processDailySalesWithoutChunking() { 1 usage

    long startTime = System.currentTimeMillis();

    List<Order> todayOrders = orderRepository.findAll();

    BigDecimal totalSales = BigDecimal.ZERO;

    for (Order order : todayOrders) {

        try {

            // Simulate heavy database/report processing
            Thread.sleep( millis: 200 );

        } catch (InterruptedException e) {

            Thread.currentThread().interrupt();

        }

        totalSales =
            totalSales.add(order.getTotalPrice());

    }

    long endTime = System.currentTimeMillis();
```

يقوم النظام بـ:

1. قراءة جميع الطلبات من قاعدة البيانات .
2. تنفيذ معالجة ثقيلة لكل طلب .
3. حساب إجمالي المبيعات .
4. قياس زمن التنفيذ .

محاكاة المعالجة الثقيلة

تم استخدام:

Thread.sleep(200);

وذلك لمحاكاة:

- استعلامات قواعد بيانات ثقيلة .
- إنشاء تقارير مالية .
- عمليات حسابية معقدة .
- تحليل بيانات المبيعات .

سبب استخدام المحاكاة

لأن المشروع الأكاديمي لا يحتوي فعليًا على ملايين السجلات أو عمليات مالية حقيقية ضخمة، تم استخدام محاكاة للأحمال العالية بهدف قياس أثر استخدام Batch Processing و Chunking على الأداء.

المرحلة الثانية: التنفيذ باستخدام Chunking

الفكرة

تم تحسين النظام عبر:

- تقسيم البيانات إلى دفعات صغيرة. Chunks
- معالجة كل Chunk بشكل مستقل .
- تشغيل عدة عمليات بالتوازي باستخدام Threads.

```

public BatchResultResponse processDailySalesWithChunking() {
    List<Order> todayOrders = orderRepository.findAll();

    int chunkSize = 5;

    List<CompletableFuture<BigDecimal>> futures =
        new ArrayList<>();

    for (int i = 0; i < todayOrders.size(); i += chunkSize) {

        List<Order> chunk = todayOrders.subList(
            i,
            Math.min(i + chunkSize, todayOrders.size())

        CompletableFuture<BigDecimal> future =
            CompletableFuture.supplyAsync(
                () -> processSalesChunk(chunk)
            );

        futures.add(future);
    }

    BigDecimal totalSales = futures.stream()
        .map(CompletableFuture::join)
        .reduce(BigDecimal.ZERO, BigDecimal::add);

    long endTime = System.currentTimeMillis();
}

```

يقوم النظام بـ:

1. قراءة جميع الطلبات .
2. تقسيم الطلبات إلى Chunks.
3. تشغيل كل Chunk داخل Thread مستقل .
4. معالجة البيانات بالتوازي . Parallel Processing
5. دمج النتائج النهائية .
6. حساب الزمن الكلي للتنفيذ .

التقنيات المستخدمة:

CompletableFuture

تم استخدام:

CompletableFuture.supplyAsync()

لتنفيذ المعالجة بشكل غير متزامن. Asynchronous Processing.

تقسيم البيانات:

تم استخدام:

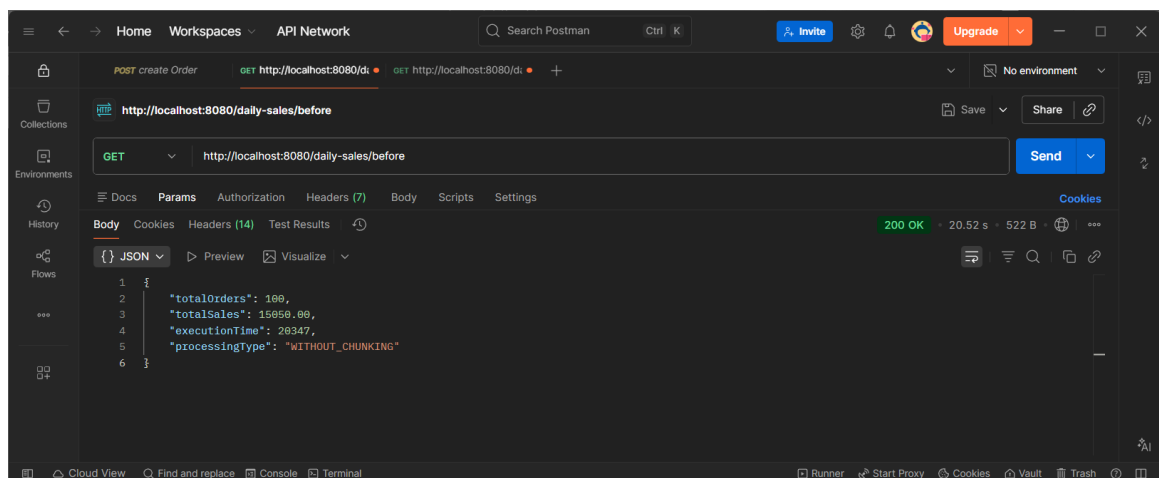
int chunkSize = 5;

أي:

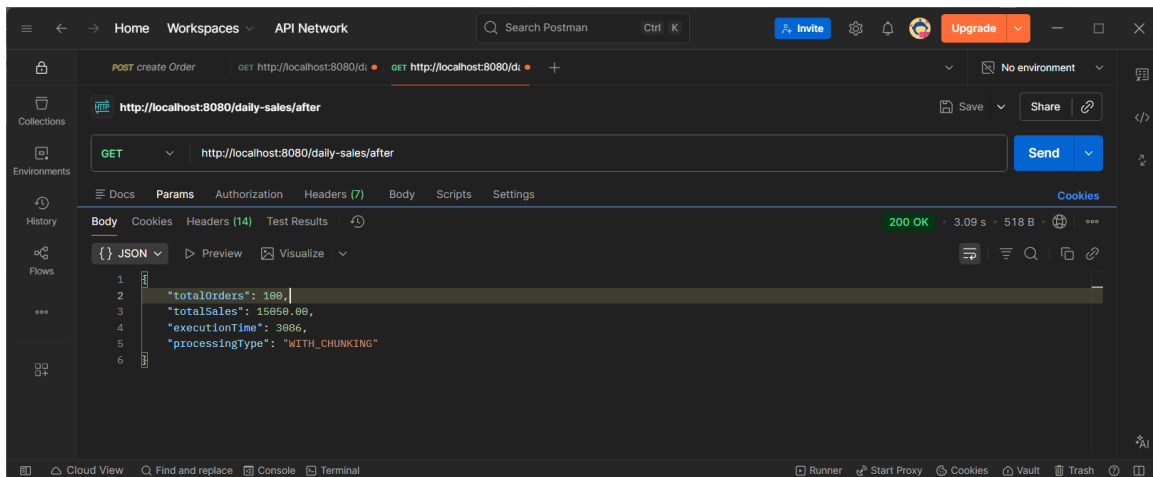
- كل Chunk يحتوي 5 Orders.
- تتم معالجة عدة Chunks بالتوازي.

API Endpoints المستخدمة:

قبل استخدام Chunking



بعد استخدام Chunking



إنشاء بيانات تم عبر

```
public class OrderSeeder implements CommandLineRunner {
    // لا تعيد الإدخال إذا موجودة بيانات
    public void run(String... args) {
        if (orderRepository.count() > 0) {
            return;
        }

        List<User> users = (List<User>) userRepository.findAll();

        if (users.isEmpty()) {
            System.out.println("❌ No users found");
            return;
        }

        for (int i = 1; i <= 100; i++) {
            Order order = new Order();

            order.setUser(users.get(i % users.size()));

            order.setOrderDate(LocalDate.now());

            order.setStatus(OrderStatus.PENDING);

            order.setTotalPrice(BigDecimal.valueOf(100 + i));

            orderRepository.save(order);
        }
    }
}
```

```
MariaDB [shoppingdb]> SELECT COUNT(*) FROM orders;
+-----+
| COUNT(*) |
+-----+
|      100 |
+-----+
1 row in set (0.027 sec)
```

النتائج

قبل

- المعالجة تتم بشكل متسلسل .
- Thread واحد فقط .
- زمن تنفيذ مرتفع .

بعد

- تقسيم البيانات إلى دفعات .
- تنفيذ متوازي باستخدام عدة Threads.
- انخفاض واضح في زمن التنفيذ .

التفسير : عند استخدام Chunking

- يتم توزيع الحمل على عدة Threads.
- تتم معالجة عدة Orders في نفس الوقت .
- يتم استغلال موارد المعالج CPU بشكل أفضل .
- يقل زمن الانتظار الكلي.

الطلب الخامس: توزيع الأحمال (Load Distribution)

1. الهدف

محاكاة توزيع الطلبات على أكثر من خادم يعمل على منافذ مختلفة، مع توضيح آلية التوزيع وتبرير استراتيجية Round Robin المستخدمة، وإثبات عملها الفعلي عبر سجلات المراقبة.

2. الاستراتيجية المستخدمة Round Robin :

وصف الاستراتيجية

تم استخدام خوارزمية **Round Robin** لتوزيع الطلبات الواردة من المستخدمين على عدة نسخ من تطبيق Spring Boot تعمل على منافذ مختلفة (8081 و 8082). يقوم موزع الأحمال (Load Balancer) الموجود في التطبيق الرئيسي على المنفذ 8080 بإرسال كل طلب قادم إلى أحد الخوادم الخلفية بالتناوب، مع حماية مؤشر التوزيع باستخدام قفل ReentrantLock لتجنب أي تضارب أثناء الإرسال المتزامن.

تبرير الاختيار

- البساطة والكفاءة: لا تحتاج Round Robin إلى قياس الحمل الفعلي على كل خادم، مما يقلل العبء على الموزع ويجعله سريعاً.
- عدالة التوزيع: تضمن توزيعاً متساوياً للطلبات على جميع الخوادم، مما يمنع إرهاق خادم واحد.
- مناسبة للنظام عديم الحالة: يعتمد المشروع على JWT للمصادقة، ولا يحتاج الطلبات إلى جلسات ثابتة على خادم معين.
- قابلية التوسع: يمكن بسهولة زيادة عدد الخوادم المضافة إلى قائمة الموزع.

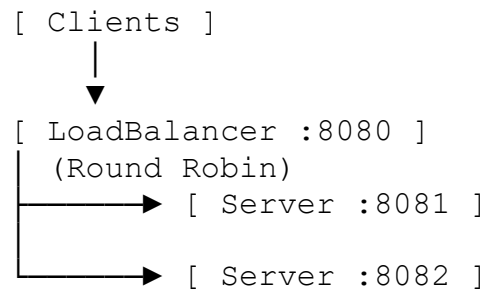
3. التصميم المعماري

يتكون نظام توزيع الأحمال من المكونات التالية:

- **(Client) العميل:** يمثل المستخدمين الذين يرسلون طلبات HTTP إلى التطبيق الرئيسي (المنفذ 8080)
- **(LoadBalance) موزع الأحمال:** مكون (Spring @Component) HttpLoadBalancer يستقبل الطلبات ويقوم بإعادة توجيهها إلى الخوادم الخلفية عبر HTTP باستخدام RestTemplate.
- **(Servers) الخوادم:** نسختان (أو أكثر) من نفس تطبيق Spring Boot تعملان على منافذ مختلفة (8081، 8082). كل نسخة قادرة على معالجة الطلبات بشكل مستقل.
- **الخدمات (Services):** داخل كل نسخة، خدمات الأعمال مثل (ProductService) التي تعالج الطلب الفعلي. لأغراض الاختبار، تم استخدام نقطة API عامة /api/public/health لا تتطلب توثيق JWT، لإثبات وصول الطلبات بنجاح.

الرسم التخطيطي:

text



4. إثبات بدء التشغيل مع عدة سيرفرات

تم تشغيل 3 نسخ من التطبيق (على المنافذ 8080، 8081، 8082)
تظهر رسالة بدء موزع الأحمال في وحدة التحكم الخاصة بالمنفذ 8080:

HTTP Load Balancer initialized with 2 servers.

```
Database dialect: MySQLDialect
Database version: 5.5.5
Default catalog/schema: shoppingdb/undefined
Autocommit mode: undefined/unknown
Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown

2026-05-08T20:38:23.370+03:00 INFO 21540 --- [shoppingCartBackend] [main] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibe
2026-05-08T20:38:23.825+03:00 INFO 21540 --- [shoppingCartBackend] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persis
2026-05-08T20:38:23.890+03:00 INFO 21540 --- [shoppingCartBackend] [main] o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath; If applicable, HQL p
2026-05-08T20:38:24.164+03:00 INFO 21540 --- [shoppingCartBackend] [main] e.AuthenticationProviderManagerConfigurer : Global AuthenticationManager configured with Au
2026-05-08T20:38:24.165+03:00 WARN 21540 --- [shoppingCartBackend] [main] r.InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with an
2026-05-08T20:38:24.545+03:00 WARN 21540 --- [shoppingCartBackend] [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default.
2026-05-08T20:38:25.194+03:00 WARN 21540 --- [shoppingCartBackend] [main] ion$DefaultTemplateResolverConfiguration : Cannot find template location: classpath:/templ
2026-05-08T20:38:25.538+03:00 INFO 21540 --- [shoppingCartBackend] [main] .w.s.a.s.AnnotationActionEndpointMapping : Supporting [WS-Addressing August 2004, WS-Addre
2026-05-08T20:38:25.596+03:00 INFO 21540 --- [shoppingCartBackend] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context
2026-05-08T20:38:25.605+03:00 INFO 21540 --- [shoppingCartBackend] [main] c.s.s.ShoppingCartBackendApplication : Started ShoppingCartBackendApplication in 7.499

HTTP Load Balancer initialized with 2 servers.
```


5. التجربة والنتائج

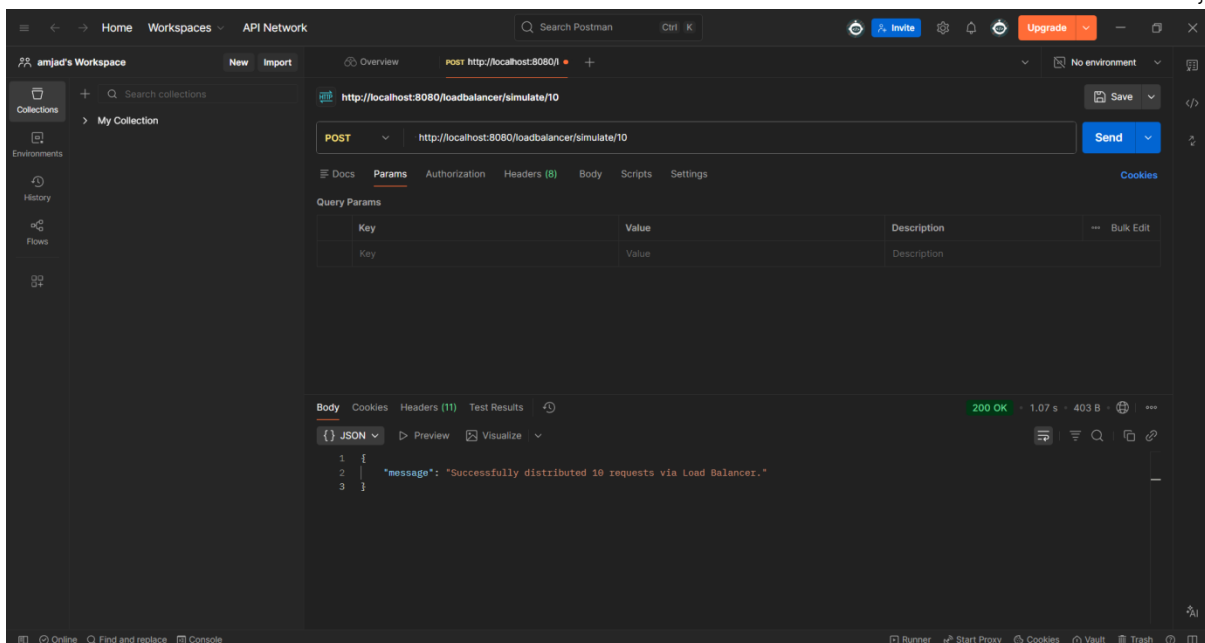
إعدادات التجربة

- عدد الخوادم الخلفية: 2 (المنفذ 8081 و 8082)
- عدد الطلبات المرسله للاختبار: 10 طلبات
- الأداة المستخدمة : Postman
- نقطة API للمستخدم POST :
`http://localhost:8080/loadbalancer/simulate/10`

نتيجة Postman

بعد إرسال الطلب، تم استقبال رد ناجح:

```
json
{
  "message": "Successfully distributed 10 requests via Load Balancer."
}
```



سجلات التوزيع (الكونسول)

في وحدة التحكم الخاصة بالخادم الرئيسي (المنفذ 8080) ظهرت السجلات التالية التي تثبت توزيع الطلبات بالتناوب بين المنفذين 8081 و 8082:

text

```
HTTP Load Balancer initialized with 2 servers.
[AOP] Forwarding request via LoadBalancer from thread: http-
nio-8080-exec-1
[LoadBalancer] Forwarding to:
http://localhost:8081/api/public/health
```

```
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-1
[LoadBalancer] Forwarding to:
http://localhost:8082/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-1
[LoadBalancer] Forwarding to:
http://localhost:8081/api/public/health
```

```
2026-05-08T20:39:04.610+03:00 INFO 21540 --- [shoppingCartBackend] [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-05-08T20:39:04.611+03:00 INFO 21540 --- [shoppingCartBackend] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-05-08T20:39:04.615+03:00 INFO 21540 --- [shoppingCartBackend] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 4 ms
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8081/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8082/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8081/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8082/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8081/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8082/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8081/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8082/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8081/api/public/health
[AOP] Forwarding request via LoadBalancer from thread: http-nio-8080-exec-2
[LoadBalancer] Forwarding to: http://localhost:8082/api/public/health
```

6. تحليل النتائج

- السجلات تظهر أن كل طلب قادم تم إرساله بالتبادل إلى المنفذين 8081 و 8082، وهذا يؤكد عمل خوارزمية Round Robin بشكل صحيح.
- ظهور العلامة [AOP] قبل كل عملية توزيع يثبت استخدام البرمجة الموجهة للجوانب (AOP) لمراقبة أداء الموزع، مما يحقق توجيهات المشروع.
- عدم حدوث أي خطأ (500 Internal Server Error) يؤكد نجاح تجاوز تحدي الحماية عبر استخدام نقطة API عامة (/api/public/health).

7. ملاحظات حول الحماية (Security)

بما أن المشروع يستخدم Spring Security مع JWT، فإن الخوادم الخلفية تحمي معظم مساراتها. لأغراض إثبات التوزيع فقط، تم إنشاء مسار عام /api/public/health غير خاضع للتوثيق، وذلك لإظهار وصول الطلبات دون الحاجة لإرسال رموز JWT. في بيئة الإنتاج، يمكن لموزع الأحمال تمرير التوكنات مع الطلبات أو استخدام مسارات محمية مع بيانات اعتماد مناسبة.

8. الخلاصة

تم تنفيذ توزيع الأحمال بنجاح باستخدام HTTP Load Balancer داخلي يعمل وفق خوارزمية Round Robin، وتم إثبات توزيع الطلبات بالتساوي بين خادمين يعملان على منافذ مختلفة، مع دمج AOP للمراقبة.

